NURa Hand-in 3

Name (Student number)

Date

For each halo-mass bin, I fit the same parametric profile (; , ,) n(x;a,b,c) separately. This yields five different best-fit parameter sets (, ,) (a,b,c), one for each mass bin. The normalization A is not an independent fit parameter, but is recalculated from , , a,b,c for each trial model. Because the radial coordinate is normalized by the virial radius, the model describes the dimensionless shape of the satellite distribution, allowing that shape to vary with halo mass.

Write a short description of your solution in each section. Explain your choices. Explicitly draw conclusions from your results.

You can refer to selected lines of code where you think it is necessary. However, you must provide the full code relevant to each (sub)question below it.

Don't forget to describe your figures and mention what conclusions you draw from them!

1 Satellite galaxies around a massive central

1.1 (1a) Maximum of N(x)

Explain the maximization algorithm used.

The maximum of $N(x)$ is found at:

$$x_{\max} \approx 0.000000$$

$$N(x_{\max}) \approx 0.000000$$

1.1.1 Code for N(x) maximization

NOTE: all code is reproduced below as an example, show only the relevant lines for this exercise here instead.

```
# imports
import matplotlib.pyplot as plt
import numpy as np


def readfile(filename):
    """
    Helper function to read in the satellite galaxy data from the provided text
    files.

    Parameters
```

```python
    ----------
    filename : str
        The name of the file to read in.

    Returns
    -------
    radius : ndarray
        The virial radius for all the satellites in the file.
    nhalo : int
        The number of halos in the file.
    """
    f = open(filename, "r")
    data = f.readlines()[3:]  # Skip first 3 lines
    nhalo = int(data[0])  # number of halos
    radius = []

    for line in data[1:]:
        if line[:-1] != "#":
            radius.append(float(line.split()[0]))

    radius = np.array(radius, dtype=float)
    f.close()
    return (
        radius,
        nhalo,
    )  # Return the virial radius for all the satellites in the file, and the number of halos


def n(x: np.ndarray, A: float, Nsat: float, a: float, b: float, c: float) -> np.ndarray:
    """
    Number density profile of satellite galaxies

    Parameters
    ----------
    x : ndarray
        Radius in units of virial radius; x = r / r_virial
    A : float
        Normalisation
    Nsat : float
        Average number of satellites
    a : float
        Small-scale slope
    b : float
        Transition scale
    c : float
        Steepness of exponential drop-off

    Returns
    -------
    ndarray
        Same type and shape as x. Number density of satellite galaxies
        at given radius x.
    """
    return 0  # insert your function (copy from hand-in 2)


# Following the lectures, the function below provides a template for a custom minimization method.
# Depending on your choice of method, you may or may not need to add more function input parameters.
```

```python
def my_minimizer(func: callable, x_arr: np.ndarray, bounds: tuple, tol: float = 1e-5) -> tuple:
    """
    Custom minimization method.

    Parameters
    ----------
    func : callable
        Function to minimize.
    x_arr : ndarray
        Array of x values to evaluate func at.
    bounds : tuple
        Tuple of (xmin, xmax) to search for minimum in.
    tol : float, optional
        Tolerance for the minimization.
        The default is 1e-5.

    Returns
    -------
    x_min : float
        Value of x at which func is minimum.
    func_min : float
        Minimum value of func.
    """

    # TODO: implement your minimization method here, e.g. by using a golden section search or Brent's method

    return 0.0, 0.0  # replace by the correct return value(s)


#### Fitting ####


def chi2(model: callable, data: np.ndarray, params: tuple) -> float:
    """
    Calculate the chi-squared for a given set of parameters and data.

    Parameters
    ----------
    model : callable
        The model function to compare to the data.
    data : ndarray
        The observed data to compare the model to.
    params : tuple
        The parameters to evaluate the model at.

    Returns
    -------
    float
        The chi-squared value for the given parameters and data.
    """
    # TODO: implement calculation of the chi2 value (or equivalent) using the model mean and variance to be minimized.

    return 0.0  # replace by the correct value


def negative_poisson_ln_likelihood(model: callable, data: np.ndarray, params: tuple) -> float:
    """
```

```python
    Calculate the Poisson negative log-likelihood for a given set of parameters and data.

    Parameters
    ----------
    model : callable
        The model function to compare to the data.
    data : ndarray
        The observed data to compare the model to.
    params : tuple
        The parameters to evaluate the model at.

    Returns
    -------
    float
        The Poisson negative log-likelihood value for the given parameters and data.
    """
    # TODO: implement calculation of the Poisson negative log-likelihood (or equivalent) to be minimized

    return 0.0  # replace by the correct value


def get_normalization_constant(a: float, b: float, c: float, Nsat: float) -> float:
    """
    Calculate the normalization constant A (which is a function of a,b,c) for the satellite number density profile.

    Parameters
    ----------
    a : float
        Small-scale slope.
    b : float
        Transition scale.
    c : float
        Steepness of exponential drop-off.
    Nsat : float
        Average number of satellites.

    Returns
    -------
    float
        Normalization constant A.
    """
    # TODO: implement the calculation of the normalization constant
    return 0.0  # replace by the correct value


def minimize_chi2(model: callable, data: np.ndarray, initial_params: tuple) -> tuple:
    """
    Minimize the chi-squared value for a given model and data.

    Parameters
    ----------
    model : callable
        The model function to compare to the data.
    data : ndarray
        The observed data to compare the model to.
    initial_params : tuple
```

```python
        Initial guess for the parameters to minimize over.

    Returns
    -------
    best_params : tuple
        The parameters that minimize the chi-squared value.
    min_chi2 : float
        The minimum chi-squared value achieved.
    """

    # TODO: implement the minimization of chi2 using your custom method. Remember to normalize for each minimization step

    best_params = initial_params
    min_chi2 = chi2(model, data, initial_params)  # replace by the correct calculation of chi2 for the given parameters

    return best_params, min_chi2


def minimize_poisson_ln_likelihood(model: callable, data: np.ndarray, initial_params: tuple) -> tuple:
    """
    Minimize the Poisson negative log-likelihood for a given model and data.

    Parameters
    ----------
    model : callable
        The model function to compare to the data.
    data : ndarray
        The observed data to compare the model to.
    initial_params : tuple
        Initial guess for the parameters to minimize over.

    Returns
    -------
    best_params : tuple
        The parameters that minimize the Poisson negative log-likelihood value.
    min_ln_likelihood : float
        The minimum Poisson negative log-likelihood value achieved.
    """

    # TODO: implement the minimization of the Poisson negative log-likelihood using your custom method. Remember to normalize for each minimization step

    best_params = initial_params
    min_ln_likelihood = negative_poisson_ln_likelihood(model, data, initial_params)  # replace by the correct calculation of the Poisson negative log-likelihood for the given parameters

    return best_params, min_ln_likelihood


# ======================================================
# ======== Main functions for each subquestion ========
# ======================================================


def do_question_1a():
    # ======== Question 1a: Maximization of N(x) ========
    a = 2.4
    b = 0.25
```

```python
    c = 1.6
    Nsat = 100
    A_1a = 256 / (5 * np.pi ** (3 / 2))
    x_lower, x_upper = 10**-4, 5

    x_max, Nx_max = my_minimizer(lambda x: 0.0, np.array([0.0]), (x_lower, x_upper))
    # replace with calculation of the maximum of N(x) based on n(x, A, Nsat, a, b, c) and your minimizer

    # Write the results to text files for later use in the PDF
    with open("Calculations/satellite_max_x.txt", "w") as f:
        f.write(f"{x_max:.6f}")
    with open("Calculations/satellite_max_Nx.txt", "w") as f:
        f.write(f"{Nx_max:.6f}")


def do_question_1b():
    # ======== Question 1b: Fitting N(x) with chi-squared ========
    datafiles = ["m11", "m12", "m13", "m14", "m15"]

    N_sat = []
    min_chi2_values = []
    best_params_chi2 = []

    # initialize figure with 5 subplots on 3x2 grid for the 5 data files
    fig, axs = plt.subplots(3, 2, figsize=(6.4, 8.0))
    axs = axs.flatten()

    for datafile in datafiles:
        radius, nhalo = readfile(f"Data/satgals_{datafile}.txt")

        x_lower, x_upper = (
            10**-4,
            5,
        )  # replace by appropriate limits for x based on the data
        bins = 10  # choose appropriate bins

        # TODO: implement the fitting of N(x) to the data using chi-squared minimization.

        # Store N_sat, chi2 values and best-fit parameters in their arrays
        N_sat.append(0.0)
        min_chi2_values.append(0.0)
        best_params_chi2.append((0.0, 0.0, 0.0))  # replace by the correct best-fit parameters (a,b,c) found from chi-squared minimization

        # Plot the data and the best-fit model for each data file in a subplot.
        axs[datafiles.index(datafile)].hist([], bins=bins)  # plot the histogram of the data

        x_plot = np.linspace(x_lower, x_upper, 100)  # create x_array for plotting the model
        axs[datafiles.index(datafile)].plot(x_plot, np.ones_like(x_plot))  # plot the best-fit model using the best-fit parameters found from chi-squared minimization

        # Add labels and title to the subplot
        axs[datafiles.index(datafile)].set_title(f"Data file: {datafile}")
        axs[datafiles.index(datafile)].set_xlabel("x = r / r_virial")
        axs[datafiles.index(datafile)].set_ylabel("Number of satellites")

```

```python
        # log-log scaling
        axs[datafiles.index(datafile)].set_xscale("log")
        axs[datafiles.index(datafile)].set_yscale("log")

    # Save the figure with all subplots
    plt.tight_layout()
    plt.savefig("Plots/satellite_fits_chi2.png")

    # Save N_sat, chi2 values and best-fit parameters for each data file to tex files for later use in the PDF
    with open("Calculations/table_fitparams_chi2.tex", "w") as f:
        rows = list(zip(N_sat, min_chi2_values, best_params_chi2))
        for idx, (N, chi2_val, params) in enumerate(rows):
            a, b, c = params
            line_end = " \\\\" if idx < len(rows) - 1 else ""
            f.write(f"m{idx + 11} & {N:.5f} & {chi2_val:.5f} & {a:.5f} & {b:.5f} & {c:.5f}{line_end}\n")


def do_question_1c():
    # ======== Question 1c: Fitting N(x) with Poisson ln-likelihood ========
    datafiles = ["m11", "m12", "m13", "m14", "m15"]

    min_poisson_llh_values = []
    best_params_poisson = []

    # initialize figure with 5 subplots on 3x2 grid for the 5 data files
    fig, axs = plt.subplots(3, 2, figsize=(6.4, 8.0))
    axs = axs.flatten()

    for datafile in datafiles:
        radius, nhalo = readfile(f"Data/satgals_{datafile}.txt")
        x_lower, x_upper = (
            10**-4,
            5,
        )  # replace by appropriate limits for x based on the data

        # TODO: implement fit using Poisson negative log-likelihood minimization.

        # Store poisson llh values and best-fit parameters in their arrays
        min_poisson_llh_values.append(0.0)
        best_params_poisson.append((0.0, 0.0, 0.0))  # replace by the correct best-fit parameters (a,b,c) found from Poisson negative log-likelihood minimization

        # Plot the data and the best-fit model for each data file in a subplot.
        axs[datafiles.index(datafile)].hist([], bins=10)  # plot the histogram of the data
        x_plot = np.linspace(x_lower, x_upper, 100)  # create x_array for plotting the model
        axs[datafiles.index(datafile)].plot(x_plot, np.ones_like(x_plot))  # plot the best-fit model using the best-fit parameters found from Poisson negative log-likelihood minimization

        # Add labels and title to the subplot
        axs[datafiles.index(datafile)].set_title(f"Data file: {datafile}")
        axs[datafiles.index(datafile)].set_xlabel("x = r / r_virial")
        axs[datafiles.index(datafile)].set_ylabel("Number of satellites")

        # log-log scaling
        axs[datafiles.index(datafile)].set_xscale("log")
        axs[datafiles.index(datafile)].set_yscale("log")
```

```python

    # Save the figure with all subplots
    plt.tight_layout()
    plt.savefig("Plots/satellite_fits_poisson.png")

    # Save poisson llh values and best-fit parameters for each data file to text files for later use in the PDF
    with open("Calculations/table_fitparams_poisson.tex", "w") as f:
        rows = list(zip(min_poisson_llh_values, best_params_poisson))
        for idx, (llh_val, params) in enumerate(rows):
            a, b, c = params
            line_end = " \\\\" if idx < len(rows) - 1 else ""
            f.write(f"m{idx + 11} & {llh_val:.5f} & {a:.5f} & {b:.5f} & {c:.5f}{line_end}\n")


def do_question_1d():
    # ======== Question 1d: Statistical tests ========
    datafiles = ["m11", "m12", "m13", "m14", "m15"]

    G_scores_chi2 = []
    Q_scores_chi2 = []

    G_scores_poisson = []
    Q_scores_poisson = []

    for datafile in datafiles:
        radius, nhalo = readfile(f"Data/satgals_{datafile}.txt")

        # Use best-fit parameters from previous steps
        best_params_chi2 = (0.0, 0.0, 0.0)  # replace by the correct array
        best_params_poisson = (0.0, 0.0, 0.0)  # replace by the correct array

        # TODO: implement the statistical tests to calculate G and Q scores for both chi2 and poisson fits, and store the results in their respective arrays

        # Append the G and Q scores for chi2 and poisson fits to their respective arrays
        G_scores_chi2.append(0.0)
        Q_scores_chi2.append(0.0)
        G_scores_poisson.append(0.0)
        Q_scores_poisson.append(0.0)

    # Save G and Q scores for chi2 and poisson fits to tex files for later use in the PDF
    with open("Calculations/statistical_test_table_rows.tex", "w") as f:
        rows = []
        for i, (G, Q) in enumerate(zip(G_scores_chi2, Q_scores_chi2), start=11):
            rows.append(f"$\\chi^2$ & m{i} & {G:.5f} & {Q:.5f}")

        for i, (G, Q) in enumerate(zip(G_scores_poisson, Q_scores_poisson), start=11):
            rows.append(f"Poisson & m{i} & {G:.5f} & {Q:.5f}")

        for idx, row in enumerate(rows):
            if idx < len(rows) - 1:
                f.write(row + " \\\\\n")
            else:
                f.write(row)


def do_question_1e():
```

```python
    # ======== Question 1e: Monte Carlo simulations ========
    # pick one of the data files to perform the Monte Carlo simulations on, e.g. m12
    datafiles = ["m11", "m12", "m13", "m14", "m15"]
    index = 1  # index of the data file to use for Monte Carlo simulations, e.g. 1 for m12

    radius, nhalo = readfile(f"Data/satgals_{datafiles[index]}.txt")

    # Use best-fit parameters from previous steps for the original data file
    best_params_chi2 = (0.0, 0.0, 0.0)  # replace by the correct array
    best_params_poisson = (0.0, 0.0, 0.0)  # replace by the correct array

    pseudo_chi2_params = []
    pseudo_poisson_params = []

    num_pseudo_experiments = 10  # replace by number with reasonable runtime
    for i in range(num_pseudo_experiments):
        # TODO: generate pseudo-data by sampling from original best-fit chi2 and poisson models
        # Then, for each pseudo-dataset, perform the chi2 and poisson fits to find the best-fit parameters.

        # Append the best-fit parameters for each pseudo-dataset to their respective arrays.
        pseudo_chi2_params.append((0.0, 0.0, 0.0))  # replace by the correct best-fit parameters (a,b,c) found from chi-squared minimization for the pseudo-dataset
        pseudo_poisson_params.append((0.0, 0.0, 0.0))  # replace by the correct best-fit parameters (a,b,c) found from Poisson negative log-likelihood minimization for the pseudo-dataset

    # plot the pseudo best-fit profiles, plot the original best-fit profile in another color and plot the mean in one more color

    # chi2 plot
    x_plot = np.linspace(1e-4, 5, 100)  # create x_array for plotting the model
    plt.figure(figsize=(6.4, 4.8))
    for params in pseudo_chi2_params:
        plt.plot(x_plot, np.ones_like(x_plot))  # plot the best-fit model for each pseudo-dataset using the best-fit parameters found from chi-squared minimization

    plt.plot(x_plot, np.ones_like(x_plot))  # plot the original best-fit model using the best-fit parameters found from chi-squared minimization on the real data

    mean_params_chi2 = np.mean(pseudo_chi2_params, axis=0)  # calculate the mean of the best-fit parameters from the pseudo-datasets
    plt.plot(x_plot, np.ones_like(x_plot))  # plot the mean of the best-fit models from the pseudo-datasets

    plt.title(f"Monte Carlo simulations - chi2 fit - Data file: {datafiles[index]}")
    plt.xlabel("x = r / r_virial")
    plt.ylabel("Number of satellites")
    plt.xscale("log")
    plt.yscale("log")
    plt.legend(["Pseudo-dataset fits", "Original fit", "Mean of pseudo fits"])
    plt.savefig("Plots/satellite_monte_carlo_chi2.png")

    # poisson plot
```

```python
    x_plot = np.linspace(1e-4, 5, 100)  # create x_array for plotting the model
    plt.figure(figsize=(6.4, 4.8))
    for params in pseudo_poisson_params:
        plt.plot(x_plot, np.ones_like(x_plot))  # plot the best-fit model for each pseudo-dataset using the best-fit parameters found from Poisson negative log-likelihood minimization
    plt.plot(x_plot, np.ones_like(x_plot))  # plot the original best-fit model using the best-fit parameters found from Poisson negative log-likelihood minimization on the real data

    mean_params_poisson = np.mean(pseudo_poisson_params, axis=0)  # calculate the mean of the best-fit parameters from the pseudo-datasets
    plt.plot(x_plot, np.ones_like(x_plot))  # plot the mean of the best-fit models from the pseudo-datasets

    plt.title(f"Monte Carlo simulations - Poisson fit - Data file: {datafiles[index]}")
    plt.xlabel("x = r / r_virial")
    plt.ylabel("Number of satellites")
    plt.xscale("log")
    plt.yscale("log")
    plt.legend(["Pseudo-dataset fits", "Original fit", "Mean of pseudo fits"])
    plt.savefig("Plots/satellite_monte_carlo_poisson.png")


if __name__ == "__main__":
    do_question_1a()
    do_question_1b()
    do_question_1c()
    do_question_1d()
    do_question_1e()
```

1.2 (1b) χ^2-fit

Argue the choices of your binning. Explain the minimization routine used. Bonus points for a correctly implemented derivative-based routine.

Table 1: Best-fit parameters for χ^2 fits.

Dataset	N_{sat}	χ^2	a	b	c
m11	0.00000	0.00000	0.00000	0.00000	0.00000
m12	0.00000	0.00000	0.00000	0.00000	0.00000
m13	0.00000	0.00000	0.00000	0.00000	0.00000
m14	0.00000	0.00000	0.00000	0.00000	0.00000
m15	0.00000	0.00000	0.00000	0.00000	0.00000

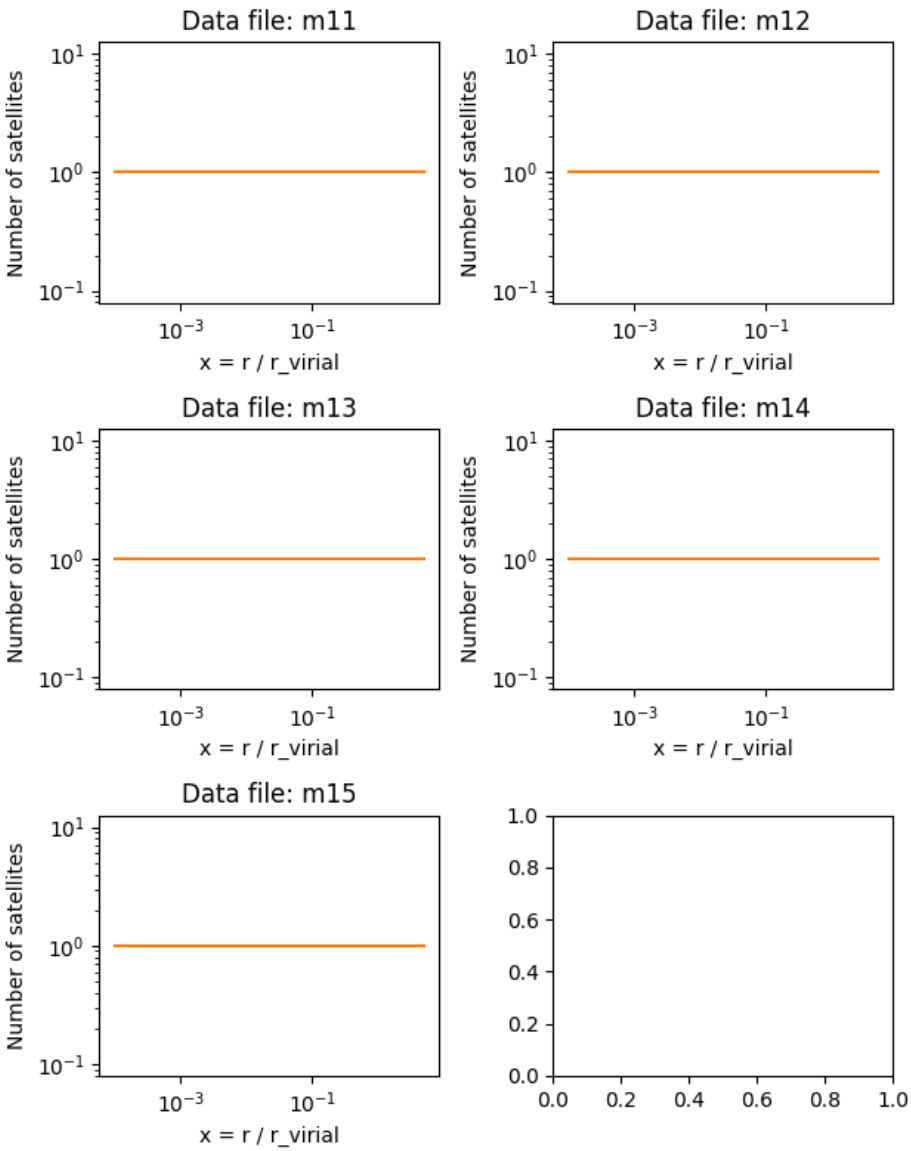

Figure 1: Fit of the satellite number density profile using the chi-squared method.

1.2.1 Code for χ^2-fit

NOTE: all code is reproduced below as an example, show only the relevant lines for this exercise here instead.

```
# imports
import matplotlib.pyplot as plt
import numpy as np


def readfile(filename):
```

```python
    """
    Helper function to read in the satellite galaxy data from the provided text files.

    Parameters
    ----------
    filename : str
        The name of the file to read in.

    Returns
    -------
    radius : ndarray
        The virial radius for all the satellites in the file.
    nhalo : int
        The number of halos in the file.
    """
    f = open(filename, "r")
    data = f.readlines()[3:]  # Skip first 3 lines
    nhalo = int(data[0])  # number of halos
    radius = []

    for line in data[1:]:
        if line[:-1] != "#":
            radius.append(float(line.split()[0]))

    radius = np.array(radius, dtype=float)
    f.close()
    return (
        radius,
        nhalo,
    )  # Return the virial radius for all the satellites in the file, and the number of halos


def n(x: np.ndarray, A: float, Nsat: float, a: float, b: float, c: float) -> np.ndarray:
    """
    Number density profile of satellite galaxies

    Parameters
    ----------
    x : ndarray
        Radius in units of virial radius; x = r / r_virial
    A : float
        Normalisation
    Nsat : float
        Average number of satellites
    a : float
        Small-scale slope
    b : float
        Transition scale
    c : float
        Steepness of exponential drop-off

    Returns
    -------
    ndarray
        Same type and shape as x. Number density of satellite galaxies
        at given radius x.
    """
    return 0  # insert your function (copy from hand-in 2)

```

```python

# Following the lectures, the function below provides a template for a custom minimization method.
# Depending on your choice of method, you may or may not need to add more function input parameters.
def my_minimizer(func: callable, x_arr: np.ndarray, bounds: tuple, tol: float = 1e-5) -> tuple:
    """
    Custom minimization method.

    Parameters
    ----------
    func : callable
        Function to minimize.
    x_arr : ndarray
        Array of x values to evaluate func at.
    bounds : tuple
        Tuple of (xmin, xmax) to search for minimum in.
    tol : float, optional
        Tolerance for the minimization.
        The default is 1e-5.

    Returns
    -------
    x_min : float
        Value of x at which func is minimum.
    func_min : float
        Minimum value of func.
    """

    # TODO: implement your minimization method here, e.g. by using a golden section search or Brent's method

    return 0.0, 0.0  # replace by the correct return value(s)


#### Fitting ####


def chi2(model: callable, data: np.ndarray, params: tuple) -> float:
    """
    Calculate the chi-squared for a given set of parameters and data.

    Parameters
    ----------
    model : callable
        The model function to compare to the data.
    data : ndarray
        The observed data to compare the model to.
    params : tuple
        The parameters to evaluate the model at.

    Returns
    -------
    float
        The chi-squared value for the given parameters and data.
    """
    # TODO: implement calculation of the chi2 value (or equivalent) using the model mean and variance to be minimized.

    return 0.0  # replace by the correct value

```

```python

def negative_poisson_ln_likelihood(model: callable, data: np.ndarray, params: tuple) -> float:
    """
    Calculate the Poisson negative log-likelihood for a given set of parameters and data.

    Parameters
    ----------
    model : callable
        The model function to compare to the data.
    data : ndarray
        The observed data to compare the model to.
    params : tuple
        The parameters to evaluate the model at.

    Returns
    -------
    float
        The Poisson negative log-likelihood value for the given parameters and data.
    """
    # TODO: implement calculation of the Poisson negative log-likelihood (or equivalent) to be minimized

    return 0.0  # replace by the correct value


def get_normalization_constant(a: float, b: float, c: float, Nsat: float) -> float:
    """
    Calculate the normalization constant A (which is a function of a,b,c) for the satellite number density profile.

    Parameters
    ----------
    a : float
        Small-scale slope.
    b : float
        Transition scale.
    c : float
        Steepness of exponential drop-off.
    Nsat : float
        Average number of satellites.

    Returns
    -------
    float
        Normalization constant A.
    """
    # TODO: implement the calculation of the normalization constant
    return 0.0  # replace by the correct value


def minimize_chi2(model: callable, data: np.ndarray, initial_params: tuple) -> tuple:
    """
    Minimize the chi-squared value for a given model and data.

    Parameters
    ----------
    model : callable
```

```python
        The model function to compare to the data.
    data : ndarray
        The observed data to compare the model to.
    initial_params : tuple
        Initial guess for the parameters to minimize over.

    Returns
    -------
    best_params : tuple
        The parameters that minimize the chi-squared value.
    min_chi2 : float
        The minimum chi-squared value achieved.
    """

    # TODO: implement the minimization of chi2 using your custom method. Remember to normalize for each minimization step

    best_params = initial_params
    min_chi2 = chi2(model, data, initial_params)  # replace by the correct calculation of chi2 for the given parameters

    return best_params, min_chi2


def minimize_poisson_ln_likelihood(model: callable, data: np.ndarray, initial_params: tuple) -> tuple:
    """
    Minimize the Poisson negative log-likelihood for a given model and data.

    Parameters
    ----------
    model : callable
        The model function to compare to the data.
    data : ndarray
        The observed data to compare the model to.
    initial_params : tuple
        Initial guess for the parameters to minimize over.

    Returns
    -------
    best_params : tuple
        The parameters that minimize the Poisson negative log-likelihood value.
    min_ln_likelihood : float
        The minimum Poisson negative log-likelihood value achieved.
    """

    # TODO: implement the minimization of the Poisson negative log-likelihood using your custom method. Remember to normalize for each minimization step

    best_params = initial_params
    min_ln_likelihood = negative_poisson_ln_likelihood(model, data, initial_params)  # replace by the correct calculation of the Poisson negative log-likelihood for the given parameters

    return best_params, min_ln_likelihood


# ======================================================
# ======== Main functions for each subquestion ========
# ======================================================


```

```python
def do_question_1a():
    # ======== Question 1a: Maximization of N(x) ========
    a = 2.4
    b = 0.25
    c = 1.6
    Nsat = 100
    A_1a = 256 / (5 * np.pi ** (3 / 2))
    x_lower, x_upper = 10**-4, 5

    x_max, Nx_max = my_minimizer(lambda x: 0.0, np.array([0.0]), (x_lower, x_upper))
    # replace with calculation of the maximum of N(x) based on n(x, A, Nsat, a, b, c) and your minimizer

    # Write the results to text files for later use in the PDF
    with open("Calculations/satellite_max_x.txt", "w") as f:
        f.write(f"{x_max:.6f}")
    with open("Calculations/satellite_max_Nx.txt", "w") as f:
        f.write(f"{Nx_max:.6f}")


def do_question_1b():
    # ======== Question 1b: Fitting N(x) with chi-squared ========
    datafiles = ["m11", "m12", "m13", "m14", "m15"]

    N_sat = []
    min_chi2_values = []
    best_params_chi2 = []

    # initialize figure with 5 subplots on 3x2 grid for the 5 data files
    fig, axs = plt.subplots(3, 2, figsize=(6.4, 8.0))
    axs = axs.flatten()

    for datafile in datafiles:
        radius, nhalo = readfile(f"Data/satgals_{datafile}.txt")

        x_lower, x_upper = (
            10**-4,
            5,
        )  # replace by appropriate limits for x based on the data
        bins = 10  # choose appropriate bins

        # TODO: implement the fitting of N(x) to the data using chi-squared minimization.

        # Store N_sat, chi2 values and best-fit parameters in their arrays
        N_sat.append(0.0)
        min_chi2_values.append(0.0)
        best_params_chi2.append((0.0, 0.0, 0.0))  # replace by the correct best-fit parameters (a,b,c) found from chi-squared minimization

        # Plot the data and the best-fit model for each data file in a subplot.
        axs[datafiles.index(datafile)].hist([], bins=bins)  # plot the histogram of the data

        x_plot = np.linspace(x_lower, x_upper, 100)  # create x_array for plotting the model
        axs[datafiles.index(datafile)].plot(x_plot, np.ones_like(x_plot))  # plot the best-fit model using the best-fit parameters found from chi-squared minimization

        # Add labels and title to the subplot
```

```python
        axs[datafiles.index(datafile)].set_title(f"Data file: {datafile}")
        axs[datafiles.index(datafile)].set_xlabel("x = r / r_virial")
        axs[datafiles.index(datafile)].set_ylabel("Number of satellites")

        # log-log scaling
        axs[datafiles.index(datafile)].set_xscale("log")
        axs[datafiles.index(datafile)].set_yscale("log")

    # Save the figure with all subplots
    plt.tight_layout()
    plt.savefig("Plots/satellite_fits_chi2.png")

    # Save N_sat, chi2 values and best-fit parameters for each data file to tex files for later use in the PDF
    with open("Calculations/table_fitparams_chi2.tex", "w") as f:
        rows = list(zip(N_sat, min_chi2_values, best_params_chi2))
        for idx, (N, chi2_val, params) in enumerate(rows):
            a, b, c = params
            line_end = " \\\\" if idx < len(rows) - 1 else ""
            f.write(f"m{idx + 11} & {N:.5f} & {chi2_val:.5f} & {a:.5f} & {b:.5f} & {c:.5f}{line_end}\n")


def do_question_1c():
    # ======== Question 1c: Fitting N(x) with Poisson ln-likelihood ========
    datafiles = ["m11", "m12", "m13", "m14", "m15"]

    min_poisson_llh_values = []
    best_params_poisson = []

    # initialize figure with 5 subplots on 3x2 grid for the 5 data files
    fig, axs = plt.subplots(3, 2, figsize=(6.4, 8.0))
    axs = axs.flatten()

    for datafile in datafiles:
        radius, nhalo = readfile(f"Data/satgals_{datafile}.txt")
        x_lower, x_upper = (
            10**-4,
            5,
        )  # replace by appropriate limits for x based on the data

        # TODO: implement fit using Poisson negative log-likelihood minimization.

        # Store poisson llh values and best-fit parameters in their arrays
        min_poisson_llh_values.append(0.0)
        best_params_poisson.append((0.0, 0.0, 0.0))  # replace by the correct best-fit parameters (a,b,c) found from Poisson negative log-likelihood minimization

        # Plot the data and the best-fit model for each data file in a subplot.
        axs[datafiles.index(datafile)].hist([], bins=10)  # plot the histogram of the data
        x_plot = np.linspace(x_lower, x_upper, 100)  # create x_array for plotting the model
        axs[datafiles.index(datafile)].plot(x_plot, np.ones_like(x_plot))  # plot the best-fit model using the best-fit parameters found from Poisson negative log-likelihood minimization

        # Add labels and title to the subplot
        axs[datafiles.index(datafile)].set_title(f"Data file: {datafile}")
        axs[datafiles.index(datafile)].set_xlabel("x = r / r_virial")
        axs[datafiles.index(datafile)].set_ylabel("Number of satellites")
```

```python

        # log-log scaling
        axs[datafiles.index(datafile)].set_xscale("log")
        axs[datafiles.index(datafile)].set_yscale("log")

    # Save the figure with all subplots
    plt.tight_layout()
    plt.savefig("Plots/satellite_fits_poisson.png")

    # Save poisson llh values and best-fit parameters for each data file to text files for later use in the PDF
    with open("Calculations/table_fitparams_poisson.tex", "w") as f:
        rows = list(zip(min_poisson_llh_values, best_params_poisson))
        for idx, (llh_val, params) in enumerate(rows):
            a, b, c = params
            line_end = " \\\\" if idx < len(rows) - 1 else ""
            f.write(f"m{idx + 11} & {llh_val:.5f} & {a:.5f} & {b:.5f} & {c:.5f}{line_end}\n")


def do_question_1d():
    # ======== Question 1d: Statistical tests ========
    datafiles = ["m11", "m12", "m13", "m14", "m15"]

    G_scores_chi2 = []
    Q_scores_chi2 = []

    G_scores_poisson = []
    Q_scores_poisson = []

    for datafile in datafiles:
        radius, nhalo = readfile(f"Data/satgals_{datafile}.txt")

        # Use best-fit parameters from previous steps
        best_params_chi2 = (0.0, 0.0, 0.0)  # replace by the correct array
        best_params_poisson = (0.0, 0.0, 0.0)  # replace by the correct array

        # TODO: implement the statistical tests to calculate G and Q scores for both chi2 and poisson fits, and store the results in their respective arrays

        # Append the G and Q scores for chi2 and poisson fits to their respective arrays
        G_scores_chi2.append(0.0)
        Q_scores_chi2.append(0.0)
        G_scores_poisson.append(0.0)
        Q_scores_poisson.append(0.0)

    # Save G and Q scores for chi2 and poisson fits to tex files for later use in the PDF
    with open("Calculations/statistical_test_table_rows.tex", "w") as f:
        rows = []
        for i, (G, Q) in enumerate(zip(G_scores_chi2, Q_scores_chi2), start=11):
            rows.append(f"$\\chi^2$ & m{i} & {G:.5f} & {Q:.5f}")

        for i, (G, Q) in enumerate(zip(G_scores_poisson, Q_scores_poisson), start=11):
            rows.append(f"Poisson & m{i} & {G:.5f} & {Q:.5f}")

        for idx, row in enumerate(rows):
            if idx < len(rows) - 1:
                f.write(row + " \\\\\n")
            else:
```

```
                f.write(row)


def do_question_1e():
    # ======== Question 1e: Monte Carlo simulations ========
    # pick one of the data files to perform the Monte Carlo simulations on, e.g. m12
    datafiles = ["m11", "m12", "m13", "m14", "m15"]
    index = 1  # index of the data file to use for Monte Carlo simulations, e.g. 1 for m12

    radius, nhalo = readfile(f"Data/satgals_{datafiles[index]}.txt")

    # Use best-fit parameters from previous steps for the original data file
    best_params_chi2 = (0.0, 0.0, 0.0)  # replace by the correct array
    best_params_poisson = (0.0, 0.0, 0.0)  # replace by the correct array

    pseudo_chi2_params = []
    pseudo_poisson_params = []

    num_pseudo_experiments = 10  # replace by number with reasonable runtime
    for i in range(num_pseudo_experiments):
        # TODO: generate pseudo-data by sampling from original best-fit chi2 and poisson models
        # Then, for each pseudo-dataset, perform the chi2 and poisson fits to find the best-fit parameters.

        # Append the best-fit parameters for each pseudo-dataset to their respective arrays.
        pseudo_chi2_params.append((0.0, 0.0, 0.0))  # replace by the correct best-fit parameters (a,b,c) found from chi-squared minimization for the pseudo-dataset
        pseudo_poisson_params.append((0.0, 0.0, 0.0))  # replace by the correct best-fit parameters (a,b,c) found from Poisson negative log-likelihood minimization for the pseudo-dataset

    # plot the pseudo best-fit profiles, plot the original best-fit profile in another color and plot the mean in one more color

    # chi2 plot
    x_plot = np.linspace(1e-4, 5, 100)  # create x_array for plotting the model
    plt.figure(figsize=(6.4, 4.8))
    for params in pseudo_chi2_params:
        plt.plot(x_plot, np.ones_like(x_plot))  # plot the best-fit model for each pseudo-dataset using the best-fit parameters found from chi-squared minimization

    plt.plot(x_plot, np.ones_like(x_plot))  # plot the original best-fit model using the best-fit parameters found from chi-squared minimization on the real data

    mean_params_chi2 = np.mean(pseudo_chi2_params, axis=0)  # calculate the mean of the best-fit parameters from the pseudo-datasets
    plt.plot(x_plot, np.ones_like(x_plot))  # plot the mean of the best-fit models from the pseudo-datasets

    plt.title(f"Monte Carlo simulations - chi2 fit - Data file: {datafiles[index]}")
    plt.xlabel("x = r / r_virial")
    plt.ylabel("Number of satellites")
    plt.xscale("log")
    plt.yscale("log")
```

```python
    plt.legend(["Pseudo-dataset fits", "Original fit", "Mean of pseudo fits"])
    plt.savefig("Plots/satellite_monte_carlo_chi2.png")

    # poisson plot
    x_plot = np.linspace(1e-4, 5, 100)  # create x_array for plotting the model
    plt.figure(figsize=(6.4, 4.8))
    for params in pseudo_poisson_params:
        plt.plot(x_plot, np.ones_like(x_plot))  # plot the best-fit model for each pseudo-dataset using the best-fit parameters found from Poisson negative log-likelihood minimization
    plt.plot(x_plot, np.ones_like(x_plot))  # plot the original best-fit model using the best-fit parameters found from Poisson negative log-likelihood minimization on the real data

    mean_params_poisson = np.mean(pseudo_poisson_params, axis=0)  # calculate the mean of the best-fit parameters from the pseudo-datasets
    plt.plot(x_plot, np.ones_like(x_plot))  # plot the mean of the best-fit models from the pseudo-datasets

    plt.title(f"Monte Carlo simulations - Poisson fit - Data file: {datafiles[index]}")
    plt.xlabel("x = r / r_virial")
    plt.ylabel("Number of satellites")
    plt.xscale("log")
    plt.yscale("log")
    plt.legend(["Pseudo-dataset fits", "Original fit", "Mean of pseudo fits"])
    plt.savefig("Plots/satellite_monte_carlo_poisson.png")


if __name__ == "__main__":
    do_question_1a()
    do_question_1b()
    do_question_1c()
    do_question_1d()
    do_question_1e()
```

1.3 (1c) Poisson ln-likelihood fit

Explain how you determine the Poisson ln-likelihood. Describe the optimization method used. How do your best fit parameters differ from the ones obtained in (1b)? Why?

Table 2: Best-fit parameters for χ^2 fits.

Dataset	llh	a	b	c
m11	0.00000	0.00000	0.00000	0.00000
m12	0.00000	0.00000	0.00000	0.00000
m13	0.00000	0.00000	0.00000	0.00000
m14	0.00000	0.00000	0.00000	0.00000
m15	0.00000	0.00000	0.00000	0.00000

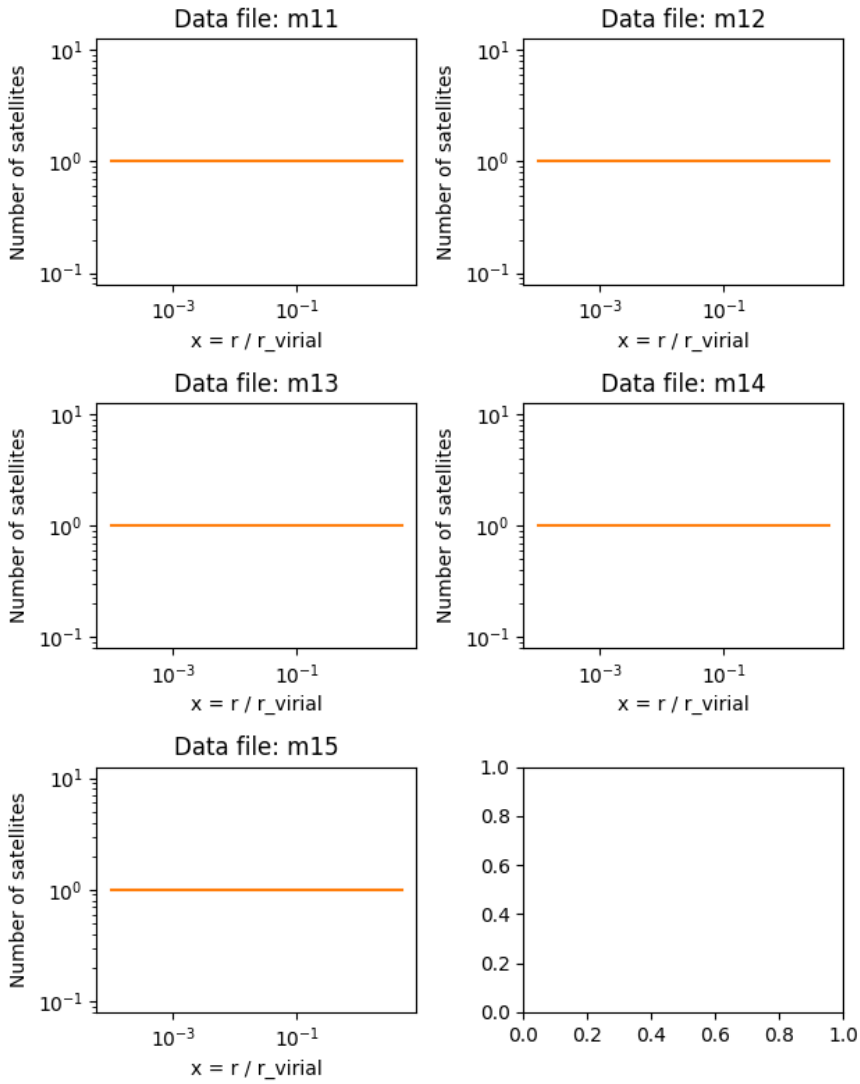

Figure 2: Fit of the satellite number density profile using the Poisson log-likelihood method.

1.3.1 Code for Poisson ln-likelihood fit

NOTE: all code is reproduced below as an example, show only the relevant lines for this exercise here instead.

```
# imports
import matplotlib.pyplot as plt
import numpy as np


def readfile(filename):
    """
    Helper function to read in the satellite galaxy data from the provided text
    files.
```

```python

    Parameters
    ----------
    filename : str
        The name of the file to read in.

    Returns
    -------
    radius : ndarray
        The virial radius for all the satellites in the file.
    nhalo : int
        The number of halos in the file.
    """
    f = open(filename, "r")
    data = f.readlines()[3:]  # Skip first 3 lines
    nhalo = int(data[0])  # number of halos
    radius = []

    for line in data[1:]:
        if line[:-1] != "#":
            radius.append(float(line.split()[0]))

    radius = np.array(radius, dtype=float)
    f.close()
    return (
        radius,
        nhalo,
    )  # Return the virial radius for all the satellites in the file, and the number of halos


def n(x: np.ndarray, A: float, Nsat: float, a: float, b: float, c: float) -> np.ndarray:
    """
    Number density profile of satellite galaxies

    Parameters
    ----------
    x : ndarray
        Radius in units of virial radius; x = r / r_virial
    A : float
        Normalisation
    Nsat : float
        Average number of satellites
    a : float
        Small-scale slope
    b : float
        Transition scale
    c : float
        Steepness of exponential drop-off

    Returns
    -------
    ndarray
        Same type and shape as x. Number density of satellite galaxies
        at given radius x.
    """
    return 0  # insert your function (copy from hand-in 2)


# Following the lectures, the function below provides a template for a custom minimization method.
```

```python
# Depending on your choice of method, you may or may not need to add more function input parameters.
def my_minimizer(func: callable, x_arr: np.ndarray, bounds: tuple, tol: float = 1e-5) -> tuple:
    """
    Custom minimization method.

    Parameters
    ----------
    func : callable
        Function to minimize.
    x_arr : ndarray
        Array of x values to evaluate func at.
    bounds : tuple
        Tuple of (xmin, xmax) to search for minimum in.
    tol : float, optional
        Tolerance for the minimization.
        The default is 1e-5.

    Returns
    -------
    x_min : float
        Value of x at which func is minimum.
    func_min : float
        Minimum value of func.
    """

    # TODO: implement your minimization method here, e.g. by using a golden section search or Brent's method

    return 0.0, 0.0  # replace by the correct return value(s)


#### Fitting ####


def chi2(model: callable, data: np.ndarray, params: tuple) -> float:
    """
    Calculate the chi-squared for a given set of parameters and data.

    Parameters
    ----------
    model : callable
        The model function to compare to the data.
    data : ndarray
        The observed data to compare the model to.
    params : tuple
        The parameters to evaluate the model at.

    Returns
    -------
    float
        The chi-squared value for the given parameters and data.
    """
    # TODO: implement calculation of the chi2 value (or equivalent) using the model mean and variance to be minimized.

    return 0.0  # replace by the correct value


def negative_poisson_ln_likelihood(model: callable, data: np.ndarray, params: tuple) -> float:
```

```python
    """
    Calculate the Poisson negative log-likelihood for a given set of parameters and data.

    Parameters
    ----------
    model : callable
        The model function to compare to the data.
    data : ndarray
        The observed data to compare the model to.
    params : tuple
        The parameters to evaluate the model at.

    Returns
    -------
    float
        The Poisson negative log-likelihood value for the given parameters and data.
    """
    # TODO: implement calculation of the Poisson negative log-likelihood (or equivalent) to be minimized

    return 0.0  # replace by the correct value


def get_normalization_constant(a: float, b: float, c: float, Nsat: float) -> float:
    """
    Calculate the normalization constant A (which is a function of a,b,c) for the satellite number density profile.

    Parameters
    ----------
    a : float
        Small-scale slope.
    b : float
        Transition scale.
    c : float
        Steepness of exponential drop-off.
    Nsat : float
        Average number of satellites.

    Returns
    -------
    float
        Normalization constant A.
    """
    # TODO: implement the calculation of the normalization constant
    return 0.0  # replace by the correct value


def minimize_chi2(model: callable, data: np.ndarray, initial_params: tuple) -> tuple:
    """
    Minimize the chi-squared value for a given model and data.

    Parameters
    ----------
    model : callable
        The model function to compare to the data.
    data : ndarray
        The observed data to compare the model to.
```

```python
    initial_params : tuple
        Initial guess for the parameters to minimize over.

    Returns
    -------
    best_params : tuple
        The parameters that minimize the chi-squared value.
    min_chi2 : float
        The minimum chi-squared value achieved.
    """

    # TODO: implement the minimization of chi2 using your custom method. Remember to normalize for each minimization step

    best_params = initial_params
    min_chi2 = chi2(model, data, initial_params)  # replace by the correct calculation of chi2 for the given parameters

    return best_params, min_chi2


def minimize_poisson_ln_likelihood(model: callable, data: np.ndarray, initial_params: tuple) -> tuple:
    """
    Minimize the Poisson negative log-likelihood for a given model and data.

    Parameters
    ----------
    model : callable
        The model function to compare to the data.
    data : ndarray
        The observed data to compare the model to.
    initial_params : tuple
        Initial guess for the parameters to minimize over.

    Returns
    -------
    best_params : tuple
        The parameters that minimize the Poisson negative log-likelihood value.
    min_ln_likelihood : float
        The minimum Poisson negative log-likelihood value achieved.
    """

    # TODO: implement the minimization of the Poisson negative log-likelihood using your custom method. Remember to normalize for each minimization step

    best_params = initial_params
    min_ln_likelihood = negative_poisson_ln_likelihood(model, data, initial_params)  # replace by the correct calculation of the Poisson negative log-likelihood for the given parameters

    return best_params, min_ln_likelihood


# ====================================================
# ======== Main functions for each subquestion ========
# ====================================================


def do_question_1a():
    # ======== Question 1a: Maximization of N(x) ========
    a = 2.4
```

```python
    b = 0.25
    c = 1.6
    Nsat = 100
    A_1a = 256 / (5 * np.pi ** (3 / 2))
    x_lower, x_upper = 10**-4, 5

    x_max, Nx_max = my_minimizer(lambda x: 0.0, np.array([0.0]), (x_lower, x_upper))
    # replace with calculation of the maximum of N(x) based on n(x, A, Nsat, a, b, c) and your minimizer

    # Write the results to text files for later use in the PDF
    with open("Calculations/satellite_max_x.txt", "w") as f:
        f.write(f"{x_max:.6f}")
    with open("Calculations/satellite_max_Nx.txt", "w") as f:
        f.write(f"{Nx_max:.6f}")


def do_question_1b():
    # ======== Question 1b: Fitting N(x) with chi-squared ========
    datafiles = ["m11", "m12", "m13", "m14", "m15"]

    N_sat = []
    min_chi2_values = []
    best_params_chi2 = []

    # initialize figure with 5 subplots on 3x2 grid for the 5 data files
    fig, axs = plt.subplots(3, 2, figsize=(6.4, 8.0))
    axs = axs.flatten()

    for datafile in datafiles:
        radius, nhalo = readfile(f"Data/satgals_{datafile}.txt")

        x_lower, x_upper = (
            10**-4,
            5,
        )  # replace by appropriate limits for x based on the data
        bins = 10  # choose appropriate bins

        # TODO: implement the fitting of N(x) to the data using chi-squared minimization.

        # Store N_sat, chi2 values and best-fit parameters in their arrays
        N_sat.append(0.0)
        min_chi2_values.append(0.0)
        best_params_chi2.append((0.0, 0.0, 0.0))  # replace by the correct best-fit parameters (a,b,c) found from chi-squared minimization

        # Plot the data and the best-fit model for each data file in a subplot.
        axs[datafiles.index(datafile)].hist([], bins=bins)  # plot the histogram of the data

        x_plot = np.linspace(x_lower, x_upper, 100)  # create x_array for plotting the model
        axs[datafiles.index(datafile)].plot(x_plot, np.ones_like(x_plot))  # plot the best-fit model using the best-fit parameters found from chi-squared minimization

        # Add labels and title to the subplot
        axs[datafiles.index(datafile)].set_title(f"Data file: {datafile}")
        axs[datafiles.index(datafile)].set_xlabel("x = r / r_virial")
        axs[datafiles.index(datafile)].set_ylabel("Number of satellites")
```

```python

        # log-log scaling
        axs[datafiles.index(datafile)].set_xscale("log")
        axs[datafiles.index(datafile)].set_yscale("log")

    # Save the figure with all subplots
    plt.tight_layout()
    plt.savefig("Plots/satellite_fits_chi2.png")

    # Save N_sat, chi2 values and best-fit parameters for each data file to tex files for later use in the PDF
    with open("Calculations/table_fitparams_chi2.tex", "w") as f:
        rows = list(zip(N_sat, min_chi2_values, best_params_chi2))
        for idx, (N, chi2_val, params) in enumerate(rows):
            a, b, c = params
            line_end = " \\\\" if idx < len(rows) - 1 else ""
            f.write(f"m{idx + 11} & {N:.5f} & {chi2_val:.5f} & {a:.5f} & {b:.5f} & {c:.5f}{line_end}\n")


def do_question_1c():
    # ======== Question 1c: Fitting N(x) with Poisson ln-likelihood ========
    datafiles = ["m11", "m12", "m13", "m14", "m15"]

    min_poisson_llh_values = []
    best_params_poisson = []

    # initialize figure with 5 subplots on 3x2 grid for the 5 data files
    fig, axs = plt.subplots(3, 2, figsize=(6.4, 8.0))
    axs = axs.flatten()

    for datafile in datafiles:
        radius, nhalo = readfile(f"Data/satgals_{datafile}.txt")
        x_lower, x_upper = (
            10**-4,
            5,
        )  # replace by appropriate limits for x based on the data

        # TODO: implement fit using Poisson negative log-likelihood minimization.

        # Store poisson llh values and best-fit parameters in their arrays
        min_poisson_llh_values.append(0.0)
        best_params_poisson.append((0.0, 0.0, 0.0))  # replace by the correct best-fit parameters (a,b,c) found from Poisson negative log-likelihood minimization

        # Plot the data and the best-fit model for each data file in a subplot.
        axs[datafiles.index(datafile)].hist([], bins=10)  # plot the histogram of the data
        x_plot = np.linspace(x_lower, x_upper, 100)  # create x_array for plotting the model
        axs[datafiles.index(datafile)].plot(x_plot, np.ones_like(x_plot))  # plot the best-fit model using the best-fit parameters found from Poisson negative log-likelihood minimization

        # Add labels and title to the subplot
        axs[datafiles.index(datafile)].set_title(f"Data file: {datafile}")
        axs[datafiles.index(datafile)].set_xlabel("x = r / r_virial")
        axs[datafiles.index(datafile)].set_ylabel("Number of satellites")

        # log-log scaling
        axs[datafiles.index(datafile)].set_xscale("log")
```

```python
        axs[datafiles.index(datafile)].set_yscale("log")

    # Save the figure with all subplots
    plt.tight_layout()
    plt.savefig("Plots/satellite_fits_poisson.png")

    # Save poisson llh values and best-fit parameters for each data file to text files for later use in the PDF
    with open("Calculations/table_fitparams_poisson.tex", "w") as f:
        rows = list(zip(min_poisson_llh_values, best_params_poisson))
        for idx, (llh_val, params) in enumerate(rows):
            a, b, c = params
            line_end = " \\\\" if idx < len(rows) - 1 else ""
            f.write(f"m{idx + 11} & {llh_val:.5f} & {a:.5f} & {b:.5f} & {c:.5f}{line_end}\n")


def do_question_1d():
    # ======== Question 1d: Statistical tests ========
    datafiles = ["m11", "m12", "m13", "m14", "m15"]

    G_scores_chi2 = []
    Q_scores_chi2 = []

    G_scores_poisson = []
    Q_scores_poisson = []

    for datafile in datafiles:
        radius, nhalo = readfile(f"Data/satgals_{datafile}.txt")

        # Use best-fit parameters from previous steps
        best_params_chi2 = (0.0, 0.0, 0.0)  # replace by the correct array
        best_params_poisson = (0.0, 0.0, 0.0)  # replace by the correct array

        # TODO: implement the statistical tests to calculate G and Q scores for both chi2 and poisson fits, and store the results in their respective arrays

        # Append the G and Q scores for chi2 and poisson fits to their respective arrays
        G_scores_chi2.append(0.0)
        Q_scores_chi2.append(0.0)
        G_scores_poisson.append(0.0)
        Q_scores_poisson.append(0.0)

    # Save G and Q scores for chi2 and poisson fits to tex files for later use in the PDF
    with open("Calculations/statistical_test_table_rows.tex", "w") as f:
        rows = []
        for i, (G, Q) in enumerate(zip(G_scores_chi2, Q_scores_chi2), start=11):
            rows.append(f"$\\chi^2$ & m{i} & {G:.5f} & {Q:.5f}")

        for i, (G, Q) in enumerate(zip(G_scores_poisson, Q_scores_poisson), start=11):
            rows.append(f"Poisson & m{i} & {G:.5f} & {Q:.5f}")

        for idx, row in enumerate(rows):
            if idx < len(rows) - 1:
                f.write(row + " \\\\\n")
            else:
                f.write(row)


```

```python
def do_question_1e():
    # ======== Question 1e: Monte Carlo simulations ========
    # pick one of the data files to perform the Monte Carlo simulations on, e.g. m12
    datafiles = ["m11", "m12", "m13", "m14", "m15"]
    index = 1  # index of the data file to use for Monte Carlo simulations, e.g. 1 for m12

    radius, nhalo = readfile(f"Data/satgals_{datafiles[index]}.txt")

    # Use best-fit parameters from previous steps for the original data file
    best_params_chi2 = (0.0, 0.0, 0.0)  # replace by the correct array
    best_params_poisson = (0.0, 0.0, 0.0)  # replace by the correct array

    pseudo_chi2_params = []
    pseudo_poisson_params = []

    num_pseudo_experiments = 10  # replace by number with reasonable runtime
    for i in range(num_pseudo_experiments):
        # TODO: generate pseudo-data by sampling from original best-fit chi2 and poisson models
        # Then, for each pseudo-dataset, perform the chi2 and poisson fits to find the best-fit parameters.

        # Append the best-fit parameters for each pseudo-dataset to their respective arrays.
        pseudo_chi2_params.append((0.0, 0.0, 0.0))  # replace by the correct best-fit parameters (a,b,c) found from chi-squared minimization for the pseudo-dataset
        pseudo_poisson_params.append((0.0, 0.0, 0.0))  # replace by the correct best-fit parameters (a,b,c) found from Poisson negative log-likelihood minimization for the pseudo-dataset

    # plot the pseudo best-fit profiles, plot the original best-fit profile in another color and plot the mean in one more color

    # chi2 plot
    x_plot = np.linspace(1e-4, 5, 100)  # create x_array for plotting the model
    plt.figure(figsize=(6.4, 4.8))
    for params in pseudo_chi2_params:
        plt.plot(x_plot, np.ones_like(x_plot))  # plot the best-fit model for each pseudo-dataset using the best-fit parameters found from chi-squared minimization

    plt.plot(x_plot, np.ones_like(x_plot))  # plot the original best-fit model using the best-fit parameters found from chi-squared minimization on the real data

    mean_params_chi2 = np.mean(pseudo_chi2_params, axis=0)  # calculate the mean of the best-fit parameters from the pseudo-datasets
    plt.plot(x_plot, np.ones_like(x_plot))  # plot the mean of the best-fit models from the pseudo-datasets

    plt.title(f"Monte Carlo simulations - chi2 fit - Data file: {datafiles[index]}")
    plt.xlabel("x = r / r_virial")
    plt.ylabel("Number of satellites")
    plt.xscale("log")
    plt.yscale("log")
    plt.legend(["Pseudo-dataset fits", "Original fit", "Mean of pseudo fits"])
    plt.savefig("Plots/satellite_monte_carlo_chi2.png")

```

```python
    # poisson plot
    x_plot = np.linspace(1e-4, 5, 100)  # create x_array for plotting the model
    plt.figure(figsize=(6.4, 4.8))
    for params in pseudo_poisson_params:
        plt.plot(x_plot, np.ones_like(x_plot))  # plot the best-fit model for each pseudo-dataset using the best-fit parameters found from Poisson negative log-likelihood minimization
    plt.plot(x_plot, np.ones_like(x_plot))  # plot the original best-fit model using the best-fit parameters found from Poisson negative log-likelihood minimization on the real data

    mean_params_poisson = np.mean(pseudo_poisson_params, axis=0)  # calculate the mean of the best-fit parameters from the pseudo-datasets
    plt.plot(x_plot, np.ones_like(x_plot))  # plot the mean of the best-fit models from the pseudo-datasets

    plt.title(f"Monte Carlo simulations - Poisson fit - Data file: {datafiles[index]}")
    plt.xlabel("x = r / r_virial")
    plt.ylabel("Number of satellites")
    plt.xscale("log")
    plt.yscale("log")
    plt.legend(["Pseudo-dataset fits", "Original fit", "Mean of pseudo fits"])
    plt.savefig("Plots/satellite_monte_carlo_poisson.png")


if __name__ == "__main__":
    do_question_1a()
    do_question_1b()
    do_question_1c()
    do_question_1d()
    do_question_1e()
```

1.4 (1d) Statistical tests

Table 3: Statistical tests for the χ^2 and Poisson ln-likelihood fits.

Method	Dataset	G	Q
χ^2	m11	0.00000	0.00000
χ^2	m12	0.00000	0.00000
χ^2	m13	0.00000	0.00000
χ^2	m14	0.00000	0.00000
χ^2	m15	0.00000	0.00000
Poisson	m11	0.00000	0.00000
Poisson	m12	0.00000	0.00000
Poisson	m13	0.00000	0.00000
Poisson	m14	0.00000	0.00000
Poisson	m15	0.00000	0.00000

What can you conclude from the results of the statistical tests?

1.4.1 Code for statistical tests

NOTE: all code is reproduced below as an example, show only the relevant lines for this exercise here instead.

```python
# imports
import matplotlib.pyplot as plt
import numpy as np


def readfile(filename):
    """
    Helper function to read in the satellite galaxy data from the provided text files.

    Parameters
    ----------
    filename : str
        The name of the file to read in.

    Returns
    -------
    radius : ndarray
        The virial radius for all the satellites in the file.
    nhalo : int
        The number of halos in the file.
    """
    f = open(filename, "r")
    data = f.readlines()[3:]  # Skip first 3 lines
    nhalo = int(data[0])  # number of halos
    radius = []

    for line in data[1:]:
        if line[:-1] != "#":
            radius.append(float(line.split()[0]))

    radius = np.array(radius, dtype=float)
    f.close()
    return (
        radius,
        nhalo,
    )  # Return the virial radius for all the satellites in the file, and the number of halos


def n(x: np.ndarray, A: float, Nsat: float, a: float, b: float, c: float) -> np.ndarray:
    """
    Number density profile of satellite galaxies

    Parameters
    ----------
    x : ndarray
        Radius in units of virial radius; x = r / r_virial
    A : float
        Normalisation
    Nsat : float
        Average number of satellites
    a : float
        Small-scale slope
    b : float
        Transition scale
    c : float
        Steepness of exponential drop-off

    Returns
    -------
```

```python
    ndarray
        Same type and shape as x. Number density of satellite galaxies
        at given radius x.
    """
    return 0  # insert your function (copy from hand-in 2)


# Following the lectures, the function below provides a template for a custom minimization method.
# Depending on your choice of method, you may or may not need to add more function input parameters.
def my_minimizer(func: callable, x_arr: np.ndarray, bounds: tuple, tol: float = 1e-5) -> tuple:
    """
    Custom minimization method.

    Parameters
    ----------
    func : callable
        Function to minimize.
    x_arr : ndarray
        Array of x values to evaluate func at.
    bounds : tuple
        Tuple of (xmin, xmax) to search for minimum in.
    tol : float, optional
        Tolerance for the minimization.
        The default is 1e-5.

    Returns
    -------
    x_min : float
        Value of x at which func is minimum.
    func_min : float
        Minimum value of func.
    """

    # TODO: implement your minimization method here, e.g. by using a golden section search or Brent's method

    return 0.0, 0.0  # replace by the correct return value(s)


#### Fitting ####


def chi2(model: callable, data: np.ndarray, params: tuple) -> float:
    """
    Calculate the chi-squared for a given set of parameters and data.

    Parameters
    ----------
    model : callable
        The model function to compare to the data.
    data : ndarray
        The observed data to compare the model to.
    params : tuple
        The parameters to evaluate the model at.

    Returns
    -------
    float
        The chi-squared value for the given parameters and data.
```

```python
    """
    # TODO: implement calculation of the chi2 value (or equivalent) using the model mean and variance to be minimized.

    return 0.0  # replace by the correct value


def negative_poisson_ln_likelihood(model: callable, data: np.ndarray, params: tuple) -> float:
    """
    Calculate the Poisson negative log-likelihood for a given set of parameters and data.

    Parameters
    ----------
    model : callable
        The model function to compare to the data.
    data : ndarray
        The observed data to compare the model to.
    params : tuple
        The parameters to evaluate the model at.

    Returns
    -------
    float
        The Poisson negative log-likelihood value for the given parameters and data.
    """
    # TODO: implement calculation of the Poisson negative log-likelihood (or equivalent) to be minimized

    return 0.0  # replace by the correct value


def get_normalization_constant(a: float, b: float, c: float, Nsat: float) -> float:
    """
    Calculate the normalization constant A (which is a function of a,b,c) for the satellite number density profile.

    Parameters
    ----------
    a : float
        Small-scale slope.
    b : float
        Transition scale.
    c : float
        Steepness of exponential drop-off.
    Nsat : float
        Average number of satellites.

    Returns
    -------
    float
        Normalization constant A.
    """
    # TODO: implement the calculation of the normalization constant
    return 0.0  # replace by the correct value


def minimize_chi2(model: callable, data: np.ndarray, initial_params: tuple) -> tuple:
```

```python
    """
    Minimize the chi-squared value for a given model and data.

    Parameters
    ----------
    model : callable
        The model function to compare to the data.
    data : ndarray
        The observed data to compare the model to.
    initial_params : tuple
        Initial guess for the parameters to minimize over.

    Returns
    -------
    best_params : tuple
        The parameters that minimize the chi-squared value.
    min_chi2 : float
        The minimum chi-squared value achieved.
    """

    # TODO: implement the minimization of chi2 using your custom method. Remember to normalize for each minimization step

    best_params = initial_params
    min_chi2 = chi2(model, data, initial_params)  # replace by the correct calculation of chi2 for the given parameters

    return best_params, min_chi2


def minimize_poisson_ln_likelihood(model: callable, data: np.ndarray, initial_params: tuple) -> tuple:
    """
    Minimize the Poisson negative log-likelihood for a given model and data.

    Parameters
    ----------
    model : callable
        The model function to compare to the data.
    data : ndarray
        The observed data to compare the model to.
    initial_params : tuple
        Initial guess for the parameters to minimize over.

    Returns
    -------
    best_params : tuple
        The parameters that minimize the Poisson negative log-likelihood value.
    min_ln_likelihood : float
        The minimum Poisson negative log-likelihood value achieved.
    """

    # TODO: implement the minimization of the Poisson negative log-likelihood using your custom method. Remember to normalize for each minimization step

    best_params = initial_params
    min_ln_likelihood = negative_poisson_ln_likelihood(model, data, initial_params)  # replace by the correct calculation of the Poisson negative log-likelihood for the given parameters

    return best_params, min_ln_likelihood

```

```python

# ======================================================
# ======== Main functions for each subquestion ========
# ======================================================


def do_question_1a():
    # ======== Question 1a: Maximization of N(x) ========
    a = 2.4
    b = 0.25
    c = 1.6
    Nsat = 100
    A_1a = 256 / (5 * np.pi ** (3 / 2))
    x_lower, x_upper = 10**-4, 5

    x_max, Nx_max = my_minimizer(lambda x: 0.0, np.array([0.0]), (x_lower, x_upper))
    # replace with calculation of the maximum of N(x) based on n(x, A, Nsat, a, b, c) and your minimizer

    # Write the results to text files for later use in the PDF
    with open("Calculations/satellite_max_x.txt", "w") as f:
        f.write(f"{x_max:.6f}")
    with open("Calculations/satellite_max_Nx.txt", "w") as f:
        f.write(f"{Nx_max:.6f}")


def do_question_1b():
    # ======== Question 1b: Fitting N(x) with chi-squared ========
    datafiles = ["m11", "m12", "m13", "m14", "m15"]

    N_sat = []
    min_chi2_values = []
    best_params_chi2 = []

    # initialize figure with 5 subplots on 3x2 grid for the 5 data files
    fig, axs = plt.subplots(3, 2, figsize=(6.4, 8.0))
    axs = axs.flatten()

    for datafile in datafiles:
        radius, nhalo = readfile(f"Data/satgals_{datafile}.txt")

        x_lower, x_upper = (
            10**-4,
            5,
        )  # replace by appropriate limits for x based on the data
        bins = 10  # choose appropriate bins

        # TODO: implement the fitting of N(x) to the data using chi-squared minimization.

        # Store N_sat, chi2 values and best-fit parameters in their arrays
        N_sat.append(0.0)
        min_chi2_values.append(0.0)
        best_params_chi2.append((0.0, 0.0, 0.0))  # replace by the correct best-fit parameters (a,b,c) found from chi-squared minimization

        # Plot the data and the best-fit model for each data file in a subplot.
        axs[datafiles.index(datafile)].hist([], bins=bins)  # plot the histogram of the data

```

```python
        x_plot = np.linspace(x_lower, x_upper, 100)  # create x_array for plotting the model
        axs[datafiles.index(datafile)].plot(x_plot, np.ones_like(x_plot))  # plot the best-fit model using the best-fit parameters found from chi-squared minimization

        # Add labels and title to the subplot
        axs[datafiles.index(datafile)].set_title(f"Data file: {datafile}")
        axs[datafiles.index(datafile)].set_xlabel("x = r / r_virial")
        axs[datafiles.index(datafile)].set_ylabel("Number of satellites")

        # log-log scaling
        axs[datafiles.index(datafile)].set_xscale("log")
        axs[datafiles.index(datafile)].set_yscale("log")

    # Save the figure with all subplots
    plt.tight_layout()
    plt.savefig("Plots/satellite_fits_chi2.png")

    # Save N_sat, chi2 values and best-fit parameters for each data file to tex files for later use in the PDF
    with open("Calculations/table_fitparams_chi2.tex", "w") as f:
        rows = list(zip(N_sat, min_chi2_values, best_params_chi2))
        for idx, (N, chi2_val, params) in enumerate(rows):
            a, b, c = params
            line_end = " \\\\" if idx < len(rows) - 1 else ""
            f.write(f"m{idx + 11} & {N:.5f} & {chi2_val:.5f} & {a:.5f} & {b:.5f} & {c:.5f}{line_end}\n")


def do_question_1c():
    # ======== Question 1c: Fitting N(x) with Poisson ln-likelihood ========
    datafiles = ["m11", "m12", "m13", "m14", "m15"]

    min_poisson_llh_values = []
    best_params_poisson = []

    # initialize figure with 5 subplots on 3x2 grid for the 5 data files
    fig, axs = plt.subplots(3, 2, figsize=(6.4, 8.0))
    axs = axs.flatten()

    for datafile in datafiles:
        radius, nhalo = readfile(f"Data/satgals_{datafile}.txt")
        x_lower, x_upper = (
            10**-4,
            5,
        )  # replace by appropriate limits for x based on the data

        # TODO: implement fit using Poisson negative log-likelihood minimization.

        # Store poisson llh values and best-fit parameters in their arrays
        min_poisson_llh_values.append(0.0)
        best_params_poisson.append((0.0, 0.0, 0.0))  # replace by the correct best-fit parameters (a,b,c) found from Poisson negative log-likelihood minimization

        # Plot the data and the best-fit model for each data file in a subplot.
        axs[datafiles.index(datafile)].hist([], bins=10)  # plot the histogram of the data
        x_plot = np.linspace(x_lower, x_upper, 100)  # create x_array for plotting the model
```

```python
        axs[datafiles.index(datafile)].plot(x_plot, np.ones_like(x_plot))  # plot the best-fit model using the best-fit parameters found from Poisson negative log-likelihood minimization

        # Add labels and title to the subplot
        axs[datafiles.index(datafile)].set_title(f"Data file: {datafile}")
        axs[datafiles.index(datafile)].set_xlabel("x = r / r_virial")
        axs[datafiles.index(datafile)].set_ylabel("Number of satellites")

        # log-log scaling
        axs[datafiles.index(datafile)].set_xscale("log")
        axs[datafiles.index(datafile)].set_yscale("log")

    # Save the figure with all subplots
    plt.tight_layout()
    plt.savefig("Plots/satellite_fits_poisson.png")

    # Save poisson llh values and best-fit parameters for each data file to text files for later use in the PDF
    with open("Calculations/table_fitparams_poisson.tex", "w") as f:
        rows = list(zip(min_poisson_llh_values, best_params_poisson))
        for idx, (llh_val, params) in enumerate(rows):
            a, b, c = params
            line_end = " \\\\" if idx < len(rows) - 1 else ""
            f.write(f"m{idx + 11} & {llh_val:.5f} & {a:.5f} & {b:.5f} & {c:.5f}{line_end}\n")


def do_question_1d():
    # ======== Question 1d: Statistical tests ========
    datafiles = ["m11", "m12", "m13", "m14", "m15"]

    G_scores_chi2 = []
    Q_scores_chi2 = []

    G_scores_poisson = []
    Q_scores_poisson = []

    for datafile in datafiles:
        radius, nhalo = readfile(f"Data/satgals_{datafile}.txt")

        # Use best-fit parameters from previous steps
        best_params_chi2 = (0.0, 0.0, 0.0)  # replace by the correct array
        best_params_poisson = (0.0, 0.0, 0.0)  # replace by the correct array

        # TODO: implement the statistical tests to calculate G and Q scores for both chi2 and poisson fits, and store the results in their respective arrays

        # Append the G and Q scores for chi2 and poisson fits to their respective arrays
        G_scores_chi2.append(0.0)
        Q_scores_chi2.append(0.0)
        G_scores_poisson.append(0.0)
        Q_scores_poisson.append(0.0)

    # Save G and Q scores for chi2 and poisson fits to tex files for later use in the PDF
    with open("Calculations/statistical_test_table_rows.tex", "w") as f:
        rows = []
        for i, (G, Q) in enumerate(zip(G_scores_chi2, Q_scores_chi2), start=11):
            rows.append(f"$\\chi^2$ & m{i} & {G:.5f} & {Q:.5f}")

```

```python
        for i, (G, Q) in enumerate(zip(G_scores_poisson, Q_scores_poisson), start=11):
            rows.append(f"Poisson & m{i} & {G:.5f} & {Q:.5f}")

        for idx, row in enumerate(rows):
            if idx < len(rows) - 1:
                f.write(row + " \\\\\n")
            else:
                f.write(row)


def do_question_1e():
    # ======== Question 1e: Monte Carlo simulations ========
    # pick one of the data files to perform the Monte Carlo simulations on, e.g. m12
    datafiles = ["m11", "m12", "m13", "m14", "m15"]
    index = 1  # index of the data file to use for Monte Carlo simulations, e.g. 1 for m12

    radius, nhalo = readfile(f"Data/satgals_{datafiles[index]}.txt")

    # Use best-fit parameters from previous steps for the original data file
    best_params_chi2 = (0.0, 0.0, 0.0)  # replace by the correct array
    best_params_poisson = (0.0, 0.0, 0.0)  # replace by the correct array

    pseudo_chi2_params = []
    pseudo_poisson_params = []

    num_pseudo_experiments = 10  # replace by number with reasonable runtime
    for i in range(num_pseudo_experiments):
        # TODO: generate pseudo-data by sampling from original best-fit chi2 and poisson models
        # Then, for each pseudo-dataset, perform the chi2 and poisson fits to find the best-fit parameters.

        # Append the best-fit parameters for each pseudo-dataset to their respective arrays.
        pseudo_chi2_params.append((0.0, 0.0, 0.0))  # replace by the correct best-fit parameters (a,b,c) found from chi-squared minimization for the pseudo-dataset
        pseudo_poisson_params.append((0.0, 0.0, 0.0))  # replace by the correct best-fit parameters (a,b,c) found from Poisson negative log-likelihood minimization for the pseudo-dataset

    # plot the pseudo best-fit profiles, plot the original best-fit profile in another color and plot the mean in one more color

    # chi2 plot
    x_plot = np.linspace(1e-4, 5, 100)  # create x_array for plotting the model
    plt.figure(figsize=(6.4, 4.8))
    for params in pseudo_chi2_params:
        plt.plot(x_plot, np.ones_like(x_plot))  # plot the best-fit model for each pseudo-dataset using the best-fit parameters found from chi-squared minimization

    plt.plot(x_plot, np.ones_like(x_plot))  # plot the original best-fit model using the best-fit parameters found from chi-squared minimization on the real data

    mean_params_chi2 = np.mean(pseudo_chi2_params, axis=0)  # calculate the mean of the best-fit parameters from the pseudo-datasets
```

```python
    plt.plot(x_plot, np.ones_like(x_plot))  # plot the mean of the best-fit models from the pseudo-datasets

    plt.title(f"Monte Carlo simulations - chi2 fit - Data file: {datafiles[index]}")
    plt.xlabel("x = r / r_virial")
    plt.ylabel("Number of satellites")
    plt.xscale("log")
    plt.yscale("log")
    plt.legend(["Pseudo-dataset fits", "Original fit", "Mean of pseudo fits"])
    plt.savefig("Plots/satellite_monte_carlo_chi2.png")

    # poisson plot
    x_plot = np.linspace(1e-4, 5, 100)  # create x_array for plotting the model
    plt.figure(figsize=(6.4, 4.8))
    for params in pseudo_poisson_params:
        plt.plot(x_plot, np.ones_like(x_plot))  # plot the best-fit model for each pseudo-dataset using the best-fit parameters found from Poisson negative log-likelihood minimization
    plt.plot(x_plot, np.ones_like(x_plot))  # plot the original best-fit model using the best-fit parameters found from Poisson negative log-likelihood minimization on the real data

    mean_params_poisson = np.mean(pseudo_poisson_params, axis=0)  # calculate the mean of the best-fit parameters from the pseudo-datasets
    plt.plot(x_plot, np.ones_like(x_plot))  # plot the mean of the best-fit models from the pseudo-datasets

    plt.title(f"Monte Carlo simulations - Poisson fit - Data file: {datafiles[index]}")
    plt.xlabel("x = r / r_virial")
    plt.ylabel("Number of satellites")
    plt.xscale("log")
    plt.yscale("log")
    plt.legend(["Pseudo-dataset fits", "Original fit", "Mean of pseudo fits"])
    plt.savefig("Plots/satellite_monte_carlo_poisson.png")


if __name__ == "__main__":
    do_question_1a()
    do_question_1b()
    do_question_1c()
    do_question_1d()
    do_question_1e()
```

1.5 (1e) Bonus Monte Carlo simulation

Pick a single dataset. Describe the implementation of your Monte Carlo parametric bootstrap.

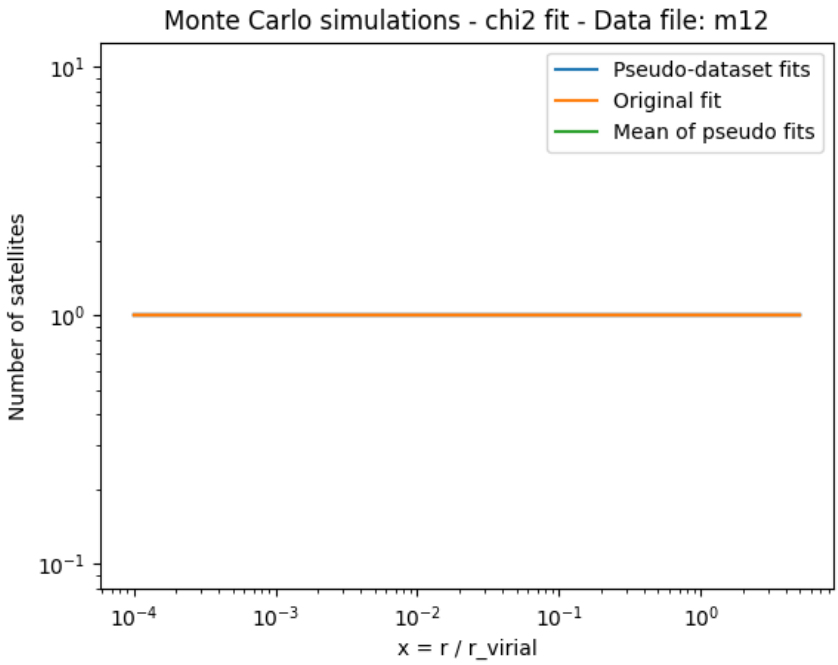

Figure 3: Best χ^2-fit profiles using pseudo-experiments.

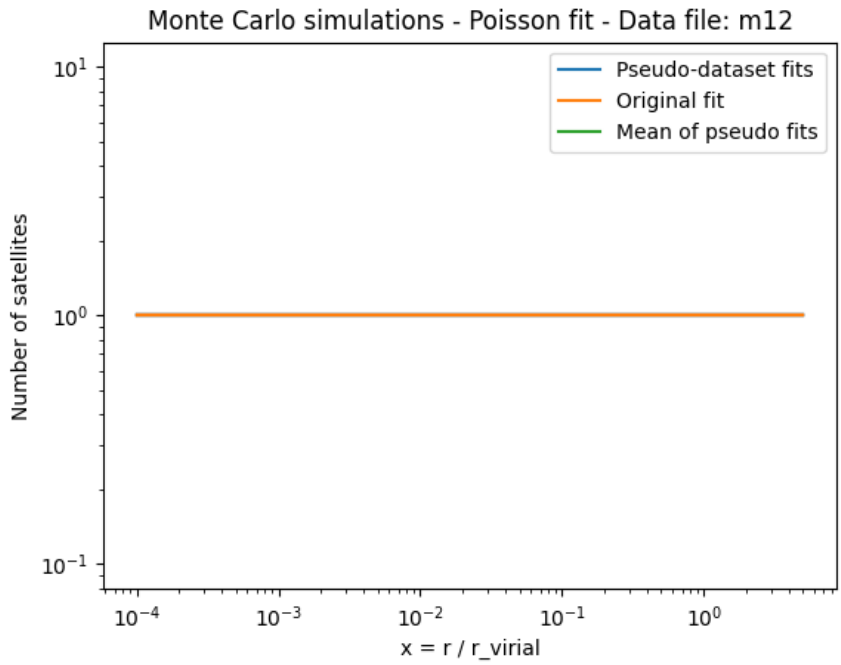

Figure 4: Best Poisson-likelihood fit profiles using pseudo-experiments.

1.5.1 Code for Monte Carlo simulation

NOTE: all code is reproduced below as an example, show only the relevant lines for this exercise here instead.

```
# imports
import matplotlib.pyplot as plt
import numpy as np


def readfile(filename):
    """
    Helper function to read in the satellite galaxy data from the provided text
    files.

    Parameters
    ----------
    filename : str
        The name of the file to read in.

    Returns
    -------
    radius : ndarray
        The virial radius for all the satellites in the file.
    nhalo : int
        The number of halos in the file.
    """
    f = open(filename, "r")
    data = f.readlines()[3:]  # Skip first 3 lines
    nhalo = int(data[0])  # number of halos
```

```python
    radius = []

    for line in data[1:]:
        if line[:-1] != "#":
            radius.append(float(line.split()[0]))

    radius = np.array(radius, dtype=float)
    f.close()
    return (
        radius,
        nhalo,
    )  # Return the virial radius for all the satellites in the file, and the number of halos


def n(x: np.ndarray, A: float, Nsat: float, a: float, b: float, c: float) -> np.ndarray:
    """
    Number density profile of satellite galaxies

    Parameters
    ----------
    x : ndarray
        Radius in units of virial radius; x = r / r_virial
    A : float
        Normalisation
    Nsat : float
        Average number of satellites
    a : float
        Small-scale slope
    b : float
        Transition scale
    c : float
        Steepness of exponential drop-off

    Returns
    -------
    ndarray
        Same type and shape as x. Number density of satellite galaxies
        at given radius x.
    """
    return 0  # insert your function (copy from hand-in 2)


# Following the lectures, the function below provides a template for a custom minimization method.
# Depending on your choice of method, you may or may not need to add more function input parameters.
def my_minimizer(func: callable, x_arr: np.ndarray, bounds: tuple, tol: float = 1e-5) -> tuple:
    """
    Custom minimization method.

    Parameters
    ----------
    func : callable
        Function to minimize.
    x_arr : ndarray
        Array of x values to evaluate func at.
    bounds : tuple
        Tuple of (xmin, xmax) to search for minimum in.
    tol : float, optional
```

```
        Tolerance for the minimization.
        The default is 1e-5.

    Returns
    -------
    x_min : float
        Value of x at which func is minimum.
    func_min : float
        Minimum value of func.
    """

    # TODO: implement your minimization method here, e.g. by using a golden section search or Brent's method

    return 0.0, 0.0  # replace by the correct return value(s)


#### Fitting ####


def chi2(model: callable, data: np.ndarray, params: tuple) -> float:
    """
    Calculate the chi-squared for a given set of parameters and data.

    Parameters
    ----------
    model : callable
        The model function to compare to the data.
    data : ndarray
        The observed data to compare the model to.
    params : tuple
        The parameters to evaluate the model at.

    Returns
    -------
    float
        The chi-squared value for the given parameters and data.
    """
    # TODO: implement calculation of the chi2 value (or equivalent) using the model mean and variance to be minimized.

    return 0.0  # replace by the correct value


def negative_poisson_ln_likelihood(model: callable, data: np.ndarray, params: tuple) -> float:
    """
    Calculate the Poisson negative log-likelihood for a given set of parameters and data.

    Parameters
    ----------
    model : callable
        The model function to compare to the data.
    data : ndarray
        The observed data to compare the model to.
    params : tuple
        The parameters to evaluate the model at.

    Returns
    -------
    float
```

```python
        The Poisson negative log-likelihood value for the given parameters and data.
    """
    # TODO: implement calculation of the Poisson negative log-likelihood (or equivalent) to be minimized

    return 0.0  # replace by the correct value


def get_normalization_constant(a: float, b: float, c: float, Nsat: float) -> float:
    """
    Calculate the normalization constant A (which is a function of a,b,c) for the satellite number density profile.

    Parameters
    ----------
    a : float
        Small-scale slope.
    b : float
        Transition scale.
    c : float
        Steepness of exponential drop-off.
    Nsat : float
        Average number of satellites.

    Returns
    -------
    float
        Normalization constant A.
    """
    # TODO: implement the calculation of the normalization constant
    return 0.0  # replace by the correct value


def minimize_chi2(model: callable, data: np.ndarray, initial_params: tuple) -> tuple:
    """
    Minimize the chi-squared value for a given model and data.

    Parameters
    ----------
    model : callable
        The model function to compare to the data.
    data : ndarray
        The observed data to compare the model to.
    initial_params : tuple
        Initial guess for the parameters to minimize over.

    Returns
    -------
    best_params : tuple
        The parameters that minimize the chi-squared value.
    min_chi2 : float
        The minimum chi-squared value achieved.
    """

    # TODO: implement the minimization of chi2 using your custom method. Remember to normalize for each minimization step

    best_params = initial_params
```

```python
    min_chi2 = chi2(model, data, initial_params)  # replace by the correct calculation of chi2 for the given parameters

    return best_params, min_chi2


def minimize_poisson_ln_likelihood(model: callable, data: np.ndarray, initial_params: tuple) -> tuple:
    """
    Minimize the Poisson negative log-likelihood for a given model and data.

    Parameters
    ----------
    model : callable
        The model function to compare to the data.
    data : ndarray
        The observed data to compare the model to.
    initial_params : tuple
        Initial guess for the parameters to minimize over.

    Returns
    -------
    best_params : tuple
        The parameters that minimize the Poisson negative log-likelihood value.
    min_ln_likelihood : float
        The minimum Poisson negative log-likelihood value achieved.
    """

    # TODO: implement the minimization of the Poisson negative log-likelihood using your custom method. Remember to normalize for each minimization step

    best_params = initial_params
    min_ln_likelihood = negative_poisson_ln_likelihood(model, data, initial_params)  # replace by the correct calculation of the Poisson negative log-likelihood for the given parameters

    return best_params, min_ln_likelihood


# ======================================================
# ======== Main functions for each subquestion ========
# ======================================================


def do_question_1a():
    # ======== Question 1a: Maximization of N(x) ========
    a = 2.4
    b = 0.25
    c = 1.6
    Nsat = 100
    A_1a = 256 / (5 * np.pi ** (3 / 2))
    x_lower, x_upper = 10**-4, 5

    x_max, Nx_max = my_minimizer(lambda x: 0.0, np.array([0.0]), (x_lower, x_upper))
    # replace with calculation of the maximum of N(x) based on n(x, A, Nsat, a, b, c) and your minimizer

    # Write the results to text files for later use in the PDF
    with open("Calculations/satellite_max_x.txt", "w") as f:
        f.write(f"{x_max:.6f}")
    with open("Calculations/satellite_max_Nx.txt", "w") as f:
```

```python
        f.write(f"{Nx_max:.6f}")


def do_question_1b():
    # ======== Question 1b: Fitting N(x) with chi-squared ========
    datafiles = ["m11", "m12", "m13", "m14", "m15"]

    N_sat = []
    min_chi2_values = []
    best_params_chi2 = []

    # initialize figure with 5 subplots on 3x2 grid for the 5 data files
    fig, axs = plt.subplots(3, 2, figsize=(6.4, 8.0))
    axs = axs.flatten()

    for datafile in datafiles:
        radius, nhalo = readfile(f"Data/satgals_{datafile}.txt")

        x_lower, x_upper = (
            10**-4,
            5,
        )  # replace by appropriate limits for x based on the data
        bins = 10  # choose appropriate bins

        # TODO: implement the fitting of N(x) to the data using chi-squared minimization.

        # Store N_sat, chi2 values and best-fit parameters in their arrays
        N_sat.append(0.0)
        min_chi2_values.append(0.0)
        best_params_chi2.append((0.0, 0.0, 0.0))  # replace by the correct best-fit parameters (a,b,c) found from chi-squared minimization

        # Plot the data and the best-fit model for each data file in a subplot.
        axs[datafiles.index(datafile)].hist([], bins=bins)  # plot the histogram of the data

        x_plot = np.linspace(x_lower, x_upper, 100)  # create x_array for plotting the model
        axs[datafiles.index(datafile)].plot(x_plot, np.ones_like(x_plot))  # plot the best-fit model using the best-fit parameters found from chi-squared minimization

        # Add labels and title to the subplot
        axs[datafiles.index(datafile)].set_title(f"Data file: {datafile}")
        axs[datafiles.index(datafile)].set_xlabel("x = r / r_virial")
        axs[datafiles.index(datafile)].set_ylabel("Number of satellites")

        # log-log scaling
        axs[datafiles.index(datafile)].set_xscale("log")
        axs[datafiles.index(datafile)].set_yscale("log")

    # Save the figure with all subplots
    plt.tight_layout()
    plt.savefig("Plots/satellite_fits_chi2.png")

    # Save N_sat, chi2 values and best-fit parameters for each data file to tex files for later use in the PDF
    with open("Calculations/table_fitparams_chi2.tex", "w") as f:
        rows = list(zip(N_sat, min_chi2_values, best_params_chi2))
        for idx, (N, chi2_val, params) in enumerate(rows):
            a, b, c = params
```

```python
            line_end = " \\\\" if idx < len(rows) - 1 else ""
            f.write(f"m{idx + 11} & {N:.5f} & {chi2_val:.5f} & {a:.5f} & {b:.5f} & {c:.5f}{line_end}\n")


def do_question_1c():
    # ======== Question 1c: Fitting N(x) with Poisson ln-likelihood ========
    datafiles = ["m11", "m12", "m13", "m14", "m15"]

    min_poisson_llh_values = []
    best_params_poisson = []

    # initialize figure with 5 subplots on 3x2 grid for the 5 data files
    fig, axs = plt.subplots(3, 2, figsize=(6.4, 8.0))
    axs = axs.flatten()

    for datafile in datafiles:
        radius, nhalo = readfile(f"Data/satgals_{datafile}.txt")
        x_lower, x_upper = (
            10**-4,
            5,
        )  # replace by appropriate limits for x based on the data

        # TODO: implement fit using Poisson negative log-likelihood minimization.

        # Store poisson llh values and best-fit parameters in their arrays
        min_poisson_llh_values.append(0.0)
        best_params_poisson.append((0.0, 0.0, 0.0))  # replace by the correct best-fit parameters (a,b,c) found from Poisson negative log-likelihood minimization

        # Plot the data and the best-fit model for each data file in a subplot.
        axs[datafiles.index(datafile)].hist([], bins=10)  # plot the histogram of the data
        x_plot = np.linspace(x_lower, x_upper, 100)  # create x_array for plotting the model
        axs[datafiles.index(datafile)].plot(x_plot, np.ones_like(x_plot))  # plot the best-fit model using the best-fit parameters found from Poisson negative log-likelihood minimization

        # Add labels and title to the subplot
        axs[datafiles.index(datafile)].set_title(f"Data file: {datafile}")
        axs[datafiles.index(datafile)].set_xlabel("x = r / r_virial")
        axs[datafiles.index(datafile)].set_ylabel("Number of satellites")

        # log-log scaling
        axs[datafiles.index(datafile)].set_xscale("log")
        axs[datafiles.index(datafile)].set_yscale("log")

    # Save the figure with all subplots
    plt.tight_layout()
    plt.savefig("Plots/satellite_fits_poisson.png")

    # Save poisson llh values and best-fit parameters for each data file to text files for later use in the PDF
    with open("Calculations/table_fitparams_poisson.tex", "w") as f:
        rows = list(zip(min_poisson_llh_values, best_params_poisson))
        for idx, (llh_val, params) in enumerate(rows):
            a, b, c = params
            line_end = " \\\\" if idx < len(rows) - 1 else ""
            f.write(f"m{idx + 11} & {llh_val:.5f} & {a:.5f} & {b:.5f} & {c:.5f}{line_end}\n")
```

```python


def do_question_1d():
    # ======== Question 1d: Statistical tests ========
    datafiles = ["m11", "m12", "m13", "m14", "m15"]

    G_scores_chi2 = []
    Q_scores_chi2 = []

    G_scores_poisson = []
    Q_scores_poisson = []

    for datafile in datafiles:
        radius, nhalo = readfile(f"Data/satgals_{datafile}.txt")

        # Use best-fit parameters from previous steps
        best_params_chi2 = (0.0, 0.0, 0.0)  # replace by the correct array
        best_params_poisson = (0.0, 0.0, 0.0)  # replace by the correct array

        # TODO: implement the statistical tests to calculate G and Q scores for both chi2 and poisson fits, and store the results in their respective arrays

        # Append the G and Q scores for chi2 and poisson fits to their respective arrays
        G_scores_chi2.append(0.0)
        Q_scores_chi2.append(0.0)
        G_scores_poisson.append(0.0)
        Q_scores_poisson.append(0.0)

    # Save G and Q scores for chi2 and poisson fits to tex files for later use in the PDF
    with open("Calculations/statistical_test_table_rows.tex", "w") as f:
        rows = []
        for i, (G, Q) in enumerate(zip(G_scores_chi2, Q_scores_chi2), start=11):
            rows.append(f"$\\chi^2$ & m{i} & {G:.5f} & {Q:.5f}")

        for i, (G, Q) in enumerate(zip(G_scores_poisson, Q_scores_poisson), start=11):
            rows.append(f"Poisson & m{i} & {G:.5f} & {Q:.5f}")

        for idx, row in enumerate(rows):
            if idx < len(rows) - 1:
                f.write(row + " \\\\\n")
            else:
                f.write(row)


def do_question_1e():
    # ======== Question 1e: Monte Carlo simulations ========
    # pick one of the data files to perform the Monte Carlo simulations on, e.g. m12
    datafiles = ["m11", "m12", "m13", "m14", "m15"]
    index = 1  # index of the data file to use for Monte Carlo simulations, e.g. 1 for m12

    radius, nhalo = readfile(f"Data/satgals_{datafiles[index]}.txt")

    # Use best-fit parameters from previous steps for the original data file
    best_params_chi2 = (0.0, 0.0, 0.0)  # replace by the correct array
    best_params_poisson = (0.0, 0.0, 0.0)  # replace by the correct array

    pseudo_chi2_params = []
```

```
    pseudo_poisson_params = []

    num_pseudo_experiments = 10  # replace by number with reasonable runtime
    for i in range(num_pseudo_experiments):
        # TODO: generate pseudo-data by sampling from original best-fit chi2 and poisson models
        # Then, for each pseudo-dataset, perform the chi2 and poisson fits to find the best-fit parameters.

        # Append the best-fit parameters for each pseudo-dataset to their respective arrays.
        pseudo_chi2_params.append((0.0, 0.0, 0.0))  # replace by the correct best-fit parameters (a,b,c) found from chi-squared minimization for the pseudo-dataset
        pseudo_poisson_params.append((0.0, 0.0, 0.0))  # replace by the correct best-fit parameters (a,b,c) found from Poisson negative log-likelihood minimization for the pseudo-dataset

    # plot the pseudo best-fit profiles, plot the original best-fit profile in another color and plot the mean in one more color

    # chi2 plot
    x_plot = np.linspace(1e-4, 5, 100)  # create x_array for plotting the model
    plt.figure(figsize=(6.4, 4.8))
    for params in pseudo_chi2_params:
        plt.plot(x_plot, np.ones_like(x_plot))  # plot the best-fit model for each pseudo-dataset using the best-fit parameters found from chi-squared minimization

    plt.plot(x_plot, np.ones_like(x_plot))  # plot the original best-fit model using the best-fit parameters found from chi-squared minimization on the real data

    mean_params_chi2 = np.mean(pseudo_chi2_params, axis=0)  # calculate the mean of the best-fit parameters from the pseudo-datasets
    plt.plot(x_plot, np.ones_like(x_plot))  # plot the mean of the best-fit models from the pseudo-datasets

    plt.title(f"Monte Carlo simulations - chi2 fit - Data file: {datafiles[index]}")
    plt.xlabel("x = r / r_virial")
    plt.ylabel("Number of satellites")
    plt.xscale("log")
    plt.yscale("log")
    plt.legend(["Pseudo-dataset fits", "Original fit", "Mean of pseudo fits"])
    plt.savefig("Plots/satellite_monte_carlo_chi2.png")

    # poisson plot
    x_plot = np.linspace(1e-4, 5, 100)  # create x_array for plotting the model
    plt.figure(figsize=(6.4, 4.8))
    for params in pseudo_poisson_params:
        plt.plot(x_plot, np.ones_like(x_plot))  # plot the best-fit model for each pseudo-dataset using the best-fit parameters found from Poisson negative log-likelihood minimization
    plt.plot(x_plot, np.ones_like(x_plot))  # plot the original best-fit model using the best-fit parameters found from Poisson negative log-likelihood minimization on the real data

    mean_params_poisson = np.mean(pseudo_poisson_params, axis=0)  # calculate the mean of the best-fit parameters from the pseudo-datasets
    plt.plot(x_plot, np.ones_like(x_plot))  # plot the mean of the best-fit models from the pseudo-datasets
```

```python

    plt.title(f"Monte Carlo simulations - Poisson fit - Data file: {datafiles[index]}")
    plt.xlabel("x = r / r_virial")
    plt.ylabel("Number of satellites")
    plt.xscale("log")
    plt.yscale("log")
    plt.legend(["Pseudo-dataset fits", "Original fit", "Mean of pseudo fits"])
    plt.savefig("Plots/satellite_monte_carlo_poisson.png")


if __name__ == "__main__":
    do_question_1a()
    do_question_1b()
    do_question_1c()
    do_question_1d()
    do_question_1e()
```